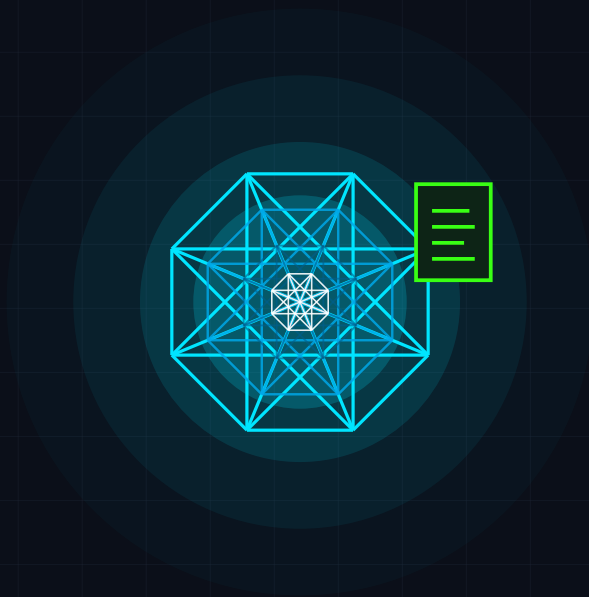




M N I M E

PROJECT ARCHITECTURE & SUBSYSTEM SCHEMATICS

Structural Topology, Neural Concurrency & Subsystem Dataflow

v5.0

Production Architecture Blueprint | Windows 11 (x64)

Lead Architect:	Kyle Bauer / kyledeanml (Bellevue College AISD)
Interface Subsystem:	PyQt6 6.7+ Frameless Obsidian Desktop Engine (60 FPS)
Document Engine:	PyMuPDF (C-Accelerated Fitz Core) & Win32 GDI Spooler
Neural Subsystem:	MNIME-Core V5 (GGUF Q4_K_M) + Neural Assimilation Engine
Vector Persistence:	FAISS Vector Store + Persistent %LOCALAPPDATA% Global Index

60 FPS Native UI

0% Network Traffic

TIES Weight Assimilation

Dual Vector Memory

Win32 Taskbar GDI

1. SYSTEM TOPOLOGY & LAYER SCHEMATIC

MNIME is engineered as a decoupled, multi-threaded desktop architecture designed for ultra-low latency, complete offline privacy, and zero telemetry. The system cleanly separates the presentation interface, document processing pipelines, neural intelligence, and semantic vector memory into four distinct functional tiers.

Layer 1: Presentation & Human Interface (PyQt6 6.7+)

Key Modules: `ui/main_window.py`, `ui/carousel_view.py`, `ui/action_bar.py`, `ui/herds.py`, `ui/file_dialog.py`

Obsidian dark metallic frameless shell. Provides 60 FPS non-blocking interaction, physics-driven particle vortex VFX, O(1) index drag-and-drop carousel ordering, custom file picker dialog, real-time performance telemetry HUD (Nerds), and magnetic detachable NLP chat console.

Layer 2: Document Processing & OS Integration Core

Key Modules: `core/pdf_engine.py`, `core/print_engine.py`, `core/app_icon.py`, `core/ipc.py`, `core/text_safety.py`

High-performance PyMuPDF (Fitz) C-level engine executing document merges, split, compress, image-to-PDF, and Word DOCX reconstruction. Includes single-instance named pipe IPC, prompt sanitization, high-DPI Windows print spooling, and Win32 taskbar registration.

Layer 3: Neural & Cognitive Subsystem

Key Modules: `core/nlp_engine.py`, `core/fusion_engine.py`, `models/MNIME-Core-V5-Q4_K_M.gguf`

Singleton inference coordinator powered by llama.cpp. Executes quantized MNIME-Core-V5 language modeling, dynamic LoRA adapter attachment, and the Neural Assimilation Engine for background asynchronous weight delta ingestion and TIES model merging.

Layer 4: Vector Memory & Semantic Indexing

Key Modules: `core/search_engine.py`, `%LOCALAPPDATA%\MNIME\global_vector_store`

Dual-tier retrieval-augmented generation (RAG). Binds in-memory FAISS indices for active documents with a persistent, cross-session global vector database. Features automatic fallback to Windows 11 OCR (WinSDK) for scanned image documents.

CONCURRENCY & THREAD ISOLATION MODEL

Heavy workloads (PyMuPDF document transformations, FAISS embedding indexing, and llama.cpp text generation) never execute on the main GUI thread. Instead, each job is wrapped in an isolated `core.worker.Worker(QThread)` instance with a thread-safe `progress_callback` signal. This architecture guarantees an uninterrupted 60 FPS UI rendering loop even during multi-gigabyte document operations.

2. COMPONENT DIRECTORY & MODULE INDEX

Every file in the repository serves a distinct operational purpose within the MNIME ecosystem. The following index details component roles, subsystem categorizations, and thread affinities.

Path / Component	Subsystem	Thread Affinity	Architectural Role
MNIME.py	Bootstrapper	Main Thread	App entry point. Applies Dark Obsidian stylesheet & instantiates MainWindow.
core/pdf_engine.py	Core Processing	Worker Thread	PyMuPDF C-level engine: Merge, Split, Compress, DOCX, JPG/TXT conversion.
core/nlp_engine.py	Neural Cognition	Worker Thread	Singleton GGUF llama.cpp inference engine with dynamic LoRA adapter hot-swapping.
core/fusion_engine.py	Neural Cognition	Async Worker	Neural Assimilation Engine: TIES weight-fusion & donor parameter absorption.
core/search_engine.py	Semantic Memory	Worker Thread	FAISS vector store, bge-small-en embeddings, and WinSDK OCR pipeline.
core/worker.py	Concurrency	Worker Thread	Generic QThread wrapper with real-time Qt progress signaling & error boundary.
core/file_item.py	Data Model	Main Thread	File item encapsulating path, metadata, thumbnail pixmap cache & status flags.
core/ipc.py	OS Integration	Main Thread	Named pipe server enforcing single-instance execution & passing file args.
core/app_icon.py	OS Integration	Main Thread	Win32 AppUserModelID taskbar registration, GDI ICO embedding & desktop links.
core/text_safety.py	Security & NLP	Synchronous	Input sanitization, prompt injection defense, and conversational fallback routing.
core/print_engine.py	OS Integration	Worker Thread	High-DPI GDI printer rendering and physical document output spooler.
ui/main_window.py	Presentation	Main Thread	Primary coordinator. Handles drag-and-drop, view routing, and window geometry.
ui/carousel_view.py	Presentation	Main Thread	Horizontal file gallery with O(1) drag-and-drop visual reordering.
ui/file_card.py	Presentation	Main Thread	Card widget displaying thumbnail, filename, progress bar, and status badge.
ui/file_dialog.py	Presentation	Main Thread	Dark metallic custom file browser replacing native OS dialog popups.
ui/nlp_view.py	Presentation	Main Thread	Conversational RAG chat interface with detachable floating translucent mode.
ui/nerds.py	Telemetry HUD	Main Thread	Real-time telemetry HUD displaying CPU, RAM, VRAM, and processing latency.
ui/merge_particles.py	VFX Animation	Main Thread (Timer)	Physics-based vortex simulation rendering glowing particle animations during tasks.
ui/action_bar.py	Presentation	Main Thread	Primary execution trigger buttons, mode actions, and master progress bar.
ui/tabs_bar.py	Presentation	Main Thread	Navigation bar hosting mode switches, NLP master toggle, and LoRA triggers.
ui/reader_dialog.py	Document Suite	Main Thread	Frameless 4.0x high-DPI document reader with smooth wheel zoom.
ui/pdf_editor.py	Document Suite	Main Thread	Visual page-by-page PDF editor (crop, rotate, delete, reorder).
training/mnime_v4 & v5*	Model Training	Cloud / Local NAE	15k V4 philosophy + 20k V5 anti-bias datasets; NAE streaming fp16 fusion.
custom_installer.py	Packaging	Main Thread	Custom standalone PyQt6 installer with flying-file visual progress bar.
MNIME.spec	Build System	PyInstaller	Specification for single-folder standalone Windows binary distribution.

3. SUBSYSTEM DEEP-DIVES & EXECUTION PIPELINES

DEEP DIVE A — THE NEURAL ASSIMILATION ENGINE (WEIGHT FUSION)

The **Neural Assimilation Engine** (`core/fusion_engine.py`) empowers MNIME to evolve dynamically beyond static model architectures. Operating on full-precision (fp16/bf16) HuggingFace safetensors directories (e.g. `training/v4_out/MNIME-Core-V4-merged`), the engine executes streaming tensor fusion:

- 1. **Parameter Delta Isolation:** The engine extracts gradient deltas relative to base matrix weights: $\Delta W = W_{donor} - W_{base}$ (optionally isolating task vectors against a common ancestor).
- 2. **TIES Sparsity Pruning:** Insignificant delta values below a density threshold are pruned to zero, preserving base model integrity.
- 3. **Sign Consensus Resolution:** For parameter conflicts across merged matrices, the engine computes majority sign consensus to eliminate destructive interference.
- 4. **Multi-Format Export:** The fused weights are written as complete safetensors folders and converted via `llama.cpp` to high-precision GGUF (Q8_0 and Q4_K_M).

DEEP DIVE B — DUAL-TIER PERSISTENT VECTOR MEMORY & OCR

Semantic intelligence in MNIME relies on a two-tier memory hierarchy coordinated by `core/search_engine.py`:

- **Tier 1 (Ephemeral Session Index):** Files queued in the active carousel are converted to text chunks and embedded into an in-memory FAISS index. This provides sub-10ms similarity search across currently loaded documents.
- **Tier 2 (Global Persistent Vector Store):** Every parsed document chunk is simultaneously indexed into a persistent global FAISS database located in `%LOCALAPPDATA%\MNIME\global_vector_store`. Chat queries synthesize knowledge across historical sessions.
- **Native WinSDK OCR Fallback:** For image-only PDFs or low-DPI scans lacking text layers, the search engine activates Windows 11's native WinSDK OCR engine, extracting clean textual layers locally with zero external cloud dependencies.

DEEP DIVE C — WINDOWS 11 INTEGRATION & ZERO-CLOUD ASSURANCE

MNIME conforms strictly to Windows 11 platform standards while maintaining absolute cryptographic data isolation:

- **Win32 Taskbar & Shell Integration:** Sets explicit `AppUserModelID` (`MNIME.Desktop`) via Win32 ctypes, preventing grouped grouping with generic `python.exe` taskbar icons and embedding native multi-resolution ICO assets.
- **Named Pipe IPC Protocol:** A local IPC server running on the main Qt thread intercepts subsequent launches of MNIME, forwarding CLI file paths to the running instance and bringing the window to foreground without spinning duplicate processes.
- **Zero Cloud Egress:** The entire execution stack (`PyMuPDF`, `llama.cpp`, FAISS, WinSDK OCR) executes in local memory with zero socket calls, zero external telemetry, and complete resistance to network sniffing.